



Polytech Clermont
Département Ingénierie Mathématiques et Data Science

Rapport de projet – Java Développement du jeu 2048

Année 2024/2025
Réalisé par : Arthur Magnier
Encadré par : Christophe de Vault

Table des matières

Introduction	5
1. Présentation du jeu.....	6
2. Conception du logiciel.....	7
2.1. Diagramme de séquence	7
2.2. Diagramme de classes	8
3. Outils utilisés	10
3.1. IntelliJ IDEA.....	10
3.2. GitHub	10
3.3. Swing.....	10
3.4. PostgreSQL	11
4. Développement du jeu	12
4.1. Classes internes	12
4.2. Fenêtres.....	12
4.3. Requêtes BDD et MVC.....	17
4.4. Layout.....	18
4.5. Factorisation du code.....	19
4.6. Classe Utils.....	20
4.7. Introduction de la langue	20
5. Architecture du projet.....	21
6. Difficultés rencontrées	22
Conclusion	23

Table des illustrations

Figure 1 : Diagramme de séquence.....	7
Figure 2 : Diagramme de classes.....	8
Figure 3 : Coordonnées dans une fenêtre Java Swing.....	12
Figure 4 : Home.....	13
Figure 5 : Login.....	13
Figure 6 : Register	14
Figure 7 : ProfileGame	14
Figure 8 : Profile	15
Figure 9 : EditProfile.....	15
Figure 10 : Game.....	16
Figure 11 : JDialogPersonalized	16
Figure 12 : Modèle utilisé pour l'application	17
Figure 13 : Illustration du BorderLayout	18
Figure 14 : Fenêtre Game avec Layout.....	19
Figure 15 : Architecture du projet.....	21

Introduction

Dans le cadre de ma formation d'ingénieur en mathématiques appliquées et data science, nous avons réalisé un projet de développement du jeu 2048 en utilisant le langage de programmation Java. Ce projet avait pour objectif de concevoir une interface graphique permettant aux utilisateurs de jouer au jeu, en s'appuyant sur la bibliothèque Java Swing. Il intégrait également une base de données PostgreSQL pour enregistrer les résultats des différentes parties jouées.

Dans ce rapport, nous commencerons par analyser la conception du jeu à l'aide de diagrammes UML. Ensuite, nous présenterons les différents outils utilisés pour le développement. Après, nous détaillerons les étapes de programmation du jeu, puis nous exposerons l'architecture globale du projet. Enfin, nous décrirons les compétences acquises grâce à ce projet ainsi que les difficultés rencontrées au cours de sa réalisation.

Le repository du projet est disponible à l'adresse suivante : <https://github.com/Arthurmgnr/2048>.

1. Présentation du jeu

Créé en 2014 par un développeur italien, le jeu 2048 est un jeu vidéo de réflexion.

Le jeu se déroule sur une grille de 4x4 cases, où des tuiles numérotées apparaissent aléatoirement. L'objectif est de combiner ces tuiles afin d'obtenir la valeur 2048, bien que le jeu puisse se poursuivre au-delà de ce seuil. Chaque tuile porte une puissance de deux (2, 4, 8, etc.), et les tuiles de même valeur fusionnent lorsqu'elles sont déplacées l'une contre l'autre, produisant une tuile de valeur doublée.

Les déplacements s'effectuent dans l'une des quatre directions (haut, bas, gauche, droite), et toutes les tuiles sur la grille se déplacent simultanément dans la direction choisie. Après chaque mouvement, une nouvelle tuile, dont la valeur peut être 2 ou 4, apparaît dans une case vide, ce qui ajoute un élément aléatoire au jeu. La partie se termine lorsque la grille est entièrement remplie et qu'aucun mouvement possible ne peut être effectué.

2. Conception du logiciel

Dans cette partie, nous allons nous intéresser à la partie de conception UML du 2048 avec l'aide de deux diagrammes.

2.1. Diagramme de séquence

Etudions d'abord le déroulement d'une partie de 2048 à l'aide du diagramme de séquence (cf. Figure 1).

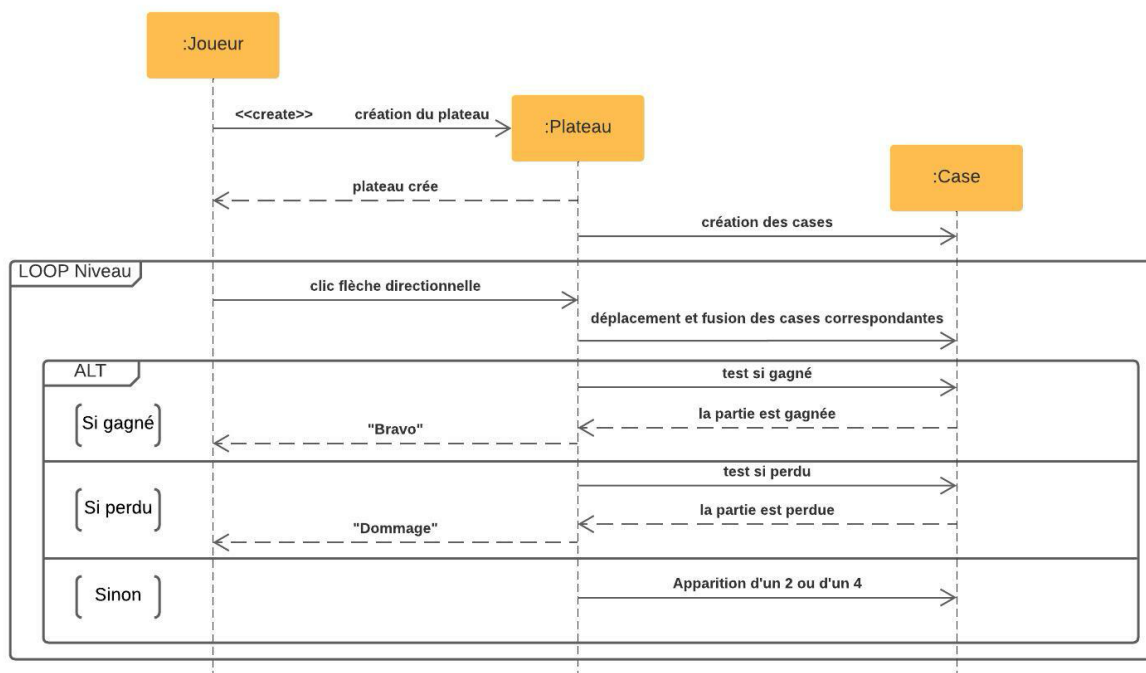


Figure 1 : Diagramme de séquence

Lorsque le joueur lance une partie de 2048, un plateau de jeu est généré, composé de 16 cases disposées en quatre lignes et quatre colonnes.

Le joueur peut alors commencer à jouer en utilisant l'une des quatre touches directionnelles du clavier. Chaque pression déclenche le déplacement et la fusion des tuiles dans la direction choisie. Une fois ces actions effectuées, trois scénarios sont possibles pour la suite de la partie :

- **Victoire** : si le joueur a atteint la tuile 2048, il remporte la partie. Un message de victoire s'affiche, et la partie prend fin automatiquement
- **Défaite** : si la grille est entièrement pleine et qu'aucun mouvement n'est possible, le joueur perd la partie. Un message de défaite s'affiche, et la partie s'arrête immédiatement
- **Continuation** : si le joueur n'a ni gagné ni perdu, une nouvelle tuile, de valeur 2 ou 4, apparaît de manière aléatoire sur une case vide, et la partie se poursuit selon la même mécanique

2.2. Diagramme de classes

Examinons à présent le diagramme de classes (cf. Figure 2), qui permet d'identifier et de structurer les objets essentiels au développement du jeu, tout en restant indépendant du langage de programmation. Cette approche facilite le développement ultérieur et permet de développer le jeu avec n'importe quel langage de programmation de manière efficace et rapide.

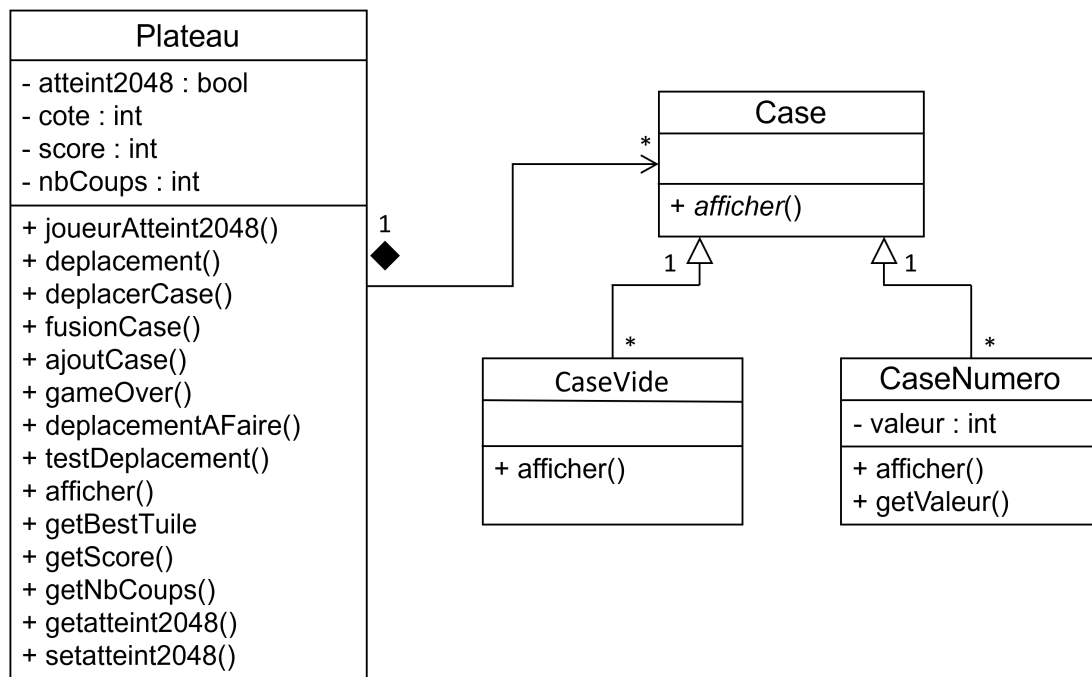


Figure 2 : Diagramme de classes

Pour représenter les cases du plateau, nous avons opté pour une hiérarchie de classes. Une classe abstraite nommée « **Case** », sert de base et définit la méthode « `afficher()` » implémentée par les deux classes qui en héritent.

Les cases du plateau se divisent en deux types distincts : celles qui contiennent une valeur (2, 4, 8, etc.) et celles qui n'en contiennent pas. Pour les représenter, nous avons utilisé les classes suivantes :

- « **CaseVide** » : cette classe représente les cases sans valeur
- « **CaseNumero** » : cette classe représente les cases contenant une valeur. Elle inclut un attribut « `valeur` » pour stocker cette donnée, ainsi que la méthode « `getValeur()` » permettant de récupérer la valeur de la case

Enfin, la classe « **Plateau** » représente le plateau de jeu et regroupe toutes les cases nécessaires au déroulement du jeu. Cette classe comprend les quatre attributs suivants :

- « `atteint2048` » : indique si le joueur a atteint la tuile 2048
- « `cote` » : stocke le nombre de lignes et de colonnes du plateau, permettant ainsi d'ajuster la taille si nécessaire
- « `score` » : enregistre le score du joueur au cours de la partie
- « `nbCoups` » : comptabilise le nombre de coups joués par le joueur pendant la partie

Elle comprend également un ensemble de méthodes, parmi lesquelles :

- Les méthodes pour obtenir ou mettre à jours les attributs : « `getScore()` », « `getNbCoups()` », « `getAtteint2048()` » et « `setAtteint2048()` »
- « `joueurAtteint2048()` » : vérifie si la tuile 2048 est présente sur le plateau
- « `deplacement()` » : gère la mécanique de déplacement après une action du joueur en appelant « `deplacerCase()` » et « `fusionCase()` »
- « `deplacerCase()` » : déplace les cases du plateau dans la direction choisie, si nécessaire
- « `fusionCase()` » : fusionne les cases lorsque cela est nécessaire
- « `ajoutCase()` » : ajoute aléatoirement une tuile de valeur 2 ou 4 sur le plateau
- « `gameOver()` » : détermine si le joueur a perdu (grille pleine et aucun déplacement possible)
- « `deplacementAFaire()` » : vérifie si un déplacement est possible dans la direction choisie
- « `testDeplacement()` » : test les différents cas possibles pour un déplacement
- « `afficher()` » : affiche le plateau avec les cases
- « `getBestTuile()` » : retourne la valeur de la tuile ayant la plus grande valeur sur le plateau

3. Outils utilisés

Dans cette partie, nous allons présenter les différents outils que nous avons utilisé pour développer le jeu 2048.

3.1. IntelliJ IDEA

IntelliJ IDEA [1] est un environnement de développement intégré (IDE en anglais), utilisé pour le développement en Java et développé par l'entreprise JetBrains [2]. Cet outil propose différentes fonctionnalités :

- **Suggestions de refactorisation** : permet de modifier le code de manière sûre et efficace avec des suggestions intelligentes
- **Complétion de code** : propose des suggestions de complétion permettant d'accélérer le développement
- **Débogage puissant** : propose des outils de débogage robustes pour identifier et corriger les erreurs rapidement

3.2. GitHub

GitHub [3] est une plateforme web dédiée à l'hébergement et à la gestion de projets logiciels. Elle fournit une interface pour utiliser **Git** [4], un système de gestion de versions. Ces outils permettent de conserver tous les fichiers d'un projet tout en enregistrant l'historique complet des modifications apportées. Cela facilite le retour à une version antérieure si nécessaire. De plus, GitHub permet le développement collaboratif sur un même projet ou, dans le cadre de ce projet, de travailler depuis deux ordinateurs différents, à domicile et à Polytech. Les fichiers étant stockés sur un serveur distant, il est possible d'y accéder depuis n'importe quel ordinateur.

Pour transférer les fichiers vers ou depuis le serveur distant, deux méthodes principales sont disponibles :

- **Ligne de commande** : en utilisant un terminal pour exécuter les différentes commandes nécessaires
- **Avec IntelliJ** : l'IDE propose des fonctionnalités intégrées permettant d'exécuter les commandes par simple clic, ce qui réduit les risques d'erreurs et accélère le processus

3.3. Swing

Swing [5] est une bibliothèque graphique intégrée au langage de programmation Java, conçue pour simplifier la création d'interfaces graphiques utilisateur. Elle offre une large gamme de composants prédéfinis pour configurer son application. Parmi ces composants figurent, par exemple, **JFrame**, utilisé pour gérer une fenêtre, et **JButton**, qui permet d'ajouter un bouton à une fenêtre.

3.4. PostgreSQL

Pour gérer les données des joueurs et des parties qu'ils jouent, nous avons utilisé **PostgreSQL** [6], un système de gestion de base de données relationnelles (SGBD) où les données sont stockées dans des tableaux bidimensionnels appelés tables.

Dans un premier temps, nous avons installé **pgAdmin4** [7], une interface permettant de visualiser et de gérer les données stockées dans les tables. Cet outil est également utilisé pour créer un serveur, où l'on peut configurer une base de données destinée à contenir les tables nécessaires au déroulement de l'application. Les informations de connexion à ce serveur sont ensuite utilisées pour interagir avec la base de données depuis l'application.

Dans un second temps, nous avons téléchargé un fichier de type « postgresql-X.X.X.jar » qui contient le **driver JDBC**. Ce fichier compressé doit être ajouté aux bibliothèques du projet depuis IntelliJ. Ce driver est essentiel, car il fournit le code permettant à l'application Java de se connecter au serveur PostgreSQL et de manipuler les données grâce aux informations de connexion.

Pour stocker les données nécessaires à l'application, nous avons utilisé deux bases de données :

- La première, **Users**, contient les données concernant les joueurs :
 - **Username** : nom d'utilisateur du joueur, doit être unique pour chaque joueur
 - **Avatar** : chemin d'accès vers l'image de profil du joueur au sein du projet
 - **Creationdate** : date à laquelle le joueur a créé son compte
 - **Lang** : langue utilisée pour l'application (français ou anglais)
- La seconde, **Games**, contient les données concernant chaque partie que les joueurs effectuent :
 - **Id** : entier généré automatiquement, permettant d'avoir des valeurs uniques
 - **Username** : nom d'utilisateur du joueur, permet de retrouver les informations du joueur dans la table Users
 - **Score** : score final de la partie, le score est calculé en additionnant les valeurs des deux tuiles qui sont fusionnées. Par exemple, si l'on fusionne deux tuiles avec la valeur 4, on obtient un score de 8
 - **Moves** : nombre total de déplacement effectué, incrémenté à chaque fois que le joueur appuie sur une touche directionnelle et que le déplacement et/ou la fusion ont pu être réalisés
 - **Besttile** : valeur de la plus grande tuile atteinte
 - **Win** : booléen valant true si le joueur a atteint 2048 et false sinon

4. Développement du jeu

Nous allons à présent examiner en détail toutes les étapes que nous avons suivies pour développer le jeu 2048.

4.1. Classes internes

Nous avons commencé par développer les classes internes du jeu, indépendamment de l'interface graphique, conformément au diagramme de classes (*cf. Figure 2*). Cette étape est essentielle, car ces classes constituent le cœur de la logique du jeu. Elles permettent de s'assurer que le fonctionnement du jeu est correct avant d'ajouter des fonctionnalités supplémentaires ou l'aspect graphique. En effet, si le code produit à ce stade ne fonctionne pas, il serait inutile de poursuivre le code, car le jeu ne serait tout simplement pas jouable.

Pour stocker le plateau contenant les cases, nous avons fait le choix d'utiliser une liste de liste, avec chaque sous-liste représentant une ligne du plateau.

Ensuite, pour réaliser le déplacement et/ou la fusion, notre logique fonctionne en trois temps : déplacement des cases dans la direction souhaitée, fusion des cases de même valeur et enfin un nouveau déplacement des cases. Nous avons réalisé un seul algorithme pour le déplacement et un seul algorithme pour la fusion, permettant de traiter les quatre possibilités : gauche, droit, bas et haut. Pour cela, nous avons une méthode permettant de définir les variables nécessaires aux algorithmes, selon la direction choisie par le joueur. Il y a les indices de départ et de fin pour parcourir les cases du plateau ainsi que la direction qui permet de se déplacer dans le plateau, par exemple si on doit déplacer les cases vers la droite, alors la valeur de la variable direction selon l'axe horizontal sera égale à +1, et -1 si le déplacement est vers la gauche. En revanche, pour la fusion, la valeur de la variable direction est l'opposé de celle du déplacement, pour respecter la logique de fusion du jeu.

4.2. Fenêtres

Après avoir vérifié que le jeu fonctionnait correctement, nous sommes passés au développement de l'interface graphique. Pour commencer, nous avons réalisé une petite maquette à l'aide de PowerPoint afin de visualiser l'aspect général souhaité. Ensuite, nous avons réalisé un code distinct pour chaque fenêtre, en y disposant manuellement les différents éléments (boutons, images, etc.). Pour cela, il suffit de spécifier les coordonnées (x, y) où placer l'élément dans la fenêtre, en sachant que le repère est défini de la façon suivante (*cf. Figure 3*).

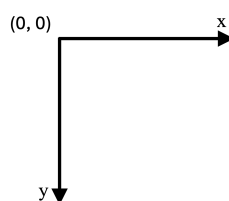


Figure 3 : Coordonnées dans une fenêtre Java Swing

Nous avons également implémenté la logique des interactions entre les différentes fenêtres via les boutons. Voici un aperçu des huit fenêtres utilisées dans l'application.

Tout d'abord, il y a la fenêtre d'accueil, appelée: Home. Le joueur a deux possibilités, selon le bouton sur lequel il clique :

- **Login** : il est alors redirigé vers la fenêtre: Login, là où il peut se connecter à son compte
- **Register** : il est alors redirigé vers la fenêtre: Register, là où il peut se créer un compte

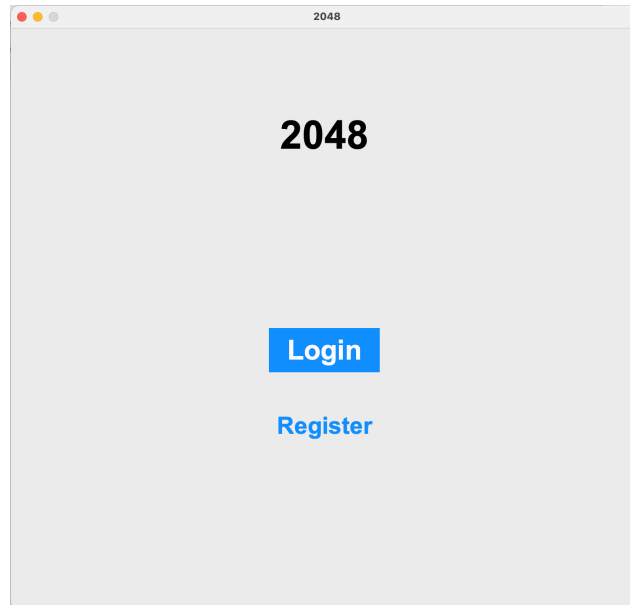


Figure 4 : Home

Ensuite, il y a la fenêtre: Login, qui permet au joueur de se connecter à son compte. Pour cela, il suffit d'inscrire son nom d'utilisateur dans la zone de texte et de cliquer sur **Login**. Si le nom d'utilisateur existe déjà, un message apparaît à l'écran lui indiquant cela. Si le nom d'utilisateur n'existe pas, il est alors redirigé vers la fenêtre: ProfileGame. Le joueur peut également revenir à la fenêtre: Home en cliquant sur le bouton **Back**.

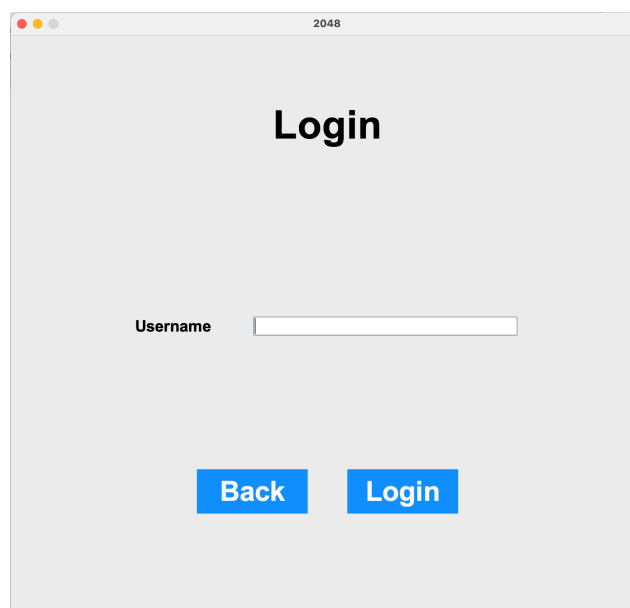


Figure 5 : Login

Puis, il y a la fenêtre: Register, qui permet au joueur de se créer un compte. Pour cela, il renseigne son nom d'utilisateur, choisit la langue qu'il souhaite utiliser pour l'application ainsi que son image de profil (avatar). Ensuite, il clique sur le bouton **Register**, si le nom d'utilisateur n'existe pas, son compte est ainsi créé et il est redirigé vers la fenêtre : ProfileGame. Si le nom d'utilisateur existe déjà, un message apparaît lui indiquant cela. Enfin, il peut également revenir vers la fenêtre: Home via le bouton **Back**.

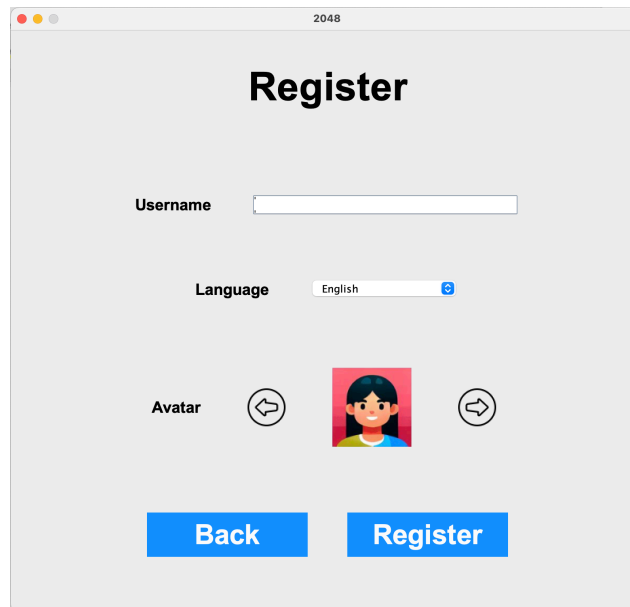
The screenshot shows a window titled "2048" with a "Register" header. Below the header, there are three input fields: "Username" with a text input, "Language" with a dropdown menu showing "English", and "Avatar" with a circular selection button and a preview of a female avatar. At the bottom, there are two blue buttons: "Back" and "Register".

Figure 6 : Register

Ensuite, il y a la fenêtre: ProfileGame, qui est une page de transition. Sur cette page, l'utilisateur a accès à son meilleur score. Il peut revenir à la page: Home en cliquant sur le bouton en haut à droite, cliquer sur le bouton **Profil** pour se rendre sur la page de consultation du profil: Profile, ou cliquer sur le bouton **Jouer** pour aller à la page: Game et ainsi jouer une partie de 2048.

The screenshot shows a window titled "2048" with a header bar containing a user icon and the name "Arthur" on the left, and a home icon on the right. The main content area displays "Meilleur score" followed by "5000" in large text. Below this, there is a blue button labeled "Jouer". At the bottom, there is a navigation bar with two buttons: "Profil" and "Jeu".

Figure 7 : ProfileGame

Puis, sur la page: Profile, le joueur consulte toutes les informations concernant à son profil. Il peut cliquer sur le bouton **Jeu** et ainsi revenir à la page: ProfileGame ou alors cliquer sur le bouton en haut à droite pour aller à la page: EditProfile afin de modifier les informations de son profil.



Figure 8 : Profile

Ensuite, sur la page: EditProfile, le joueur peut modifier la langue utilisée pour l'application ainsi que son avatar. Dès lors qu'il clique sur l'un des deux boutons, il est redirigé vers la page: Profile. La différence est que le bouton **Enregistrer** sauvegarde les modifications dans la base de données, tandis que le bouton **Retour** ne sauvegarde pas les modifications.

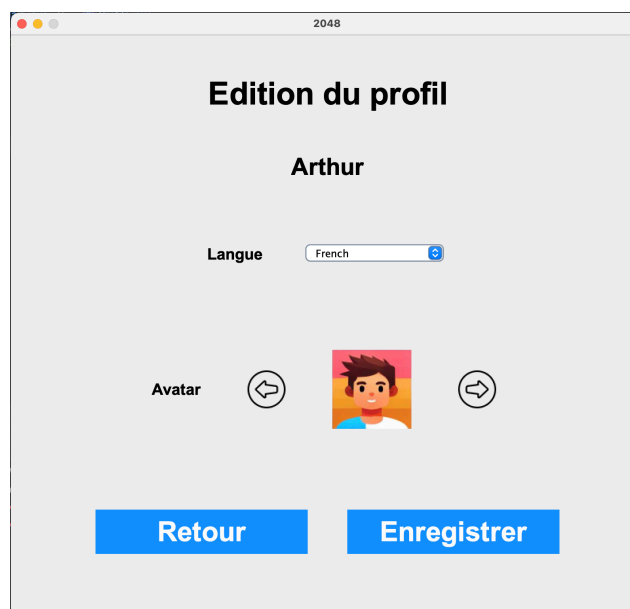


Figure 9 : EditProfile

Sur la page: Game, le joueur peut jouer une partie de 2048, en cliquant sur les touches directionnelles, et il a accès à son score actuel, son meilleur score ainsi que le nombre de mouvements. Le joueur peut, à tout moment, arrêter la partie en cliquant sur le bouton en haut à droite, et revenir à la page: ProfileGame.

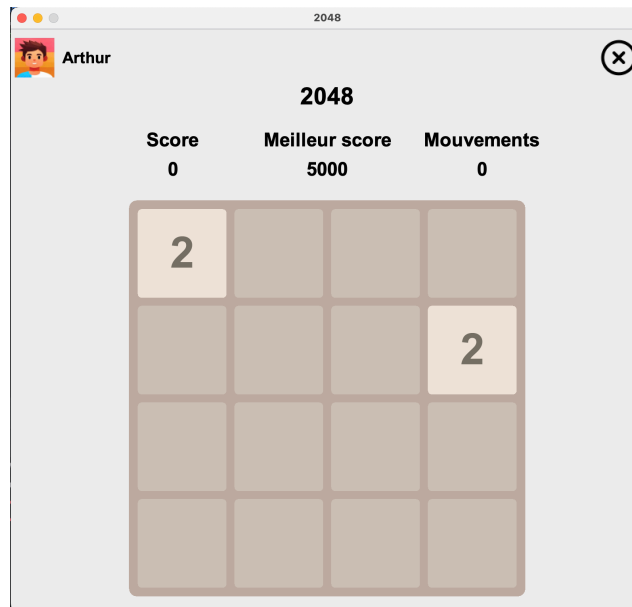


Figure 10 : Game

Ici, c'est la fenêtre: : JDialogPersonalized, qui demande au joueur une confirmation. Par exemple, cette image apparaît lorsque le joueur n'a plus aucun mouvement possible et qu'il n'a pas atteint la tuile 2048. Il a alors deux possibilités : cliquer sur le bouton **Quitter** et ainsi revenir à la fenêtre: ProfileGame, ou cliquer sur le bouton **Recommencer** et ainsi rejouer une partie. Cette fenêtre étant utilisée à trois reprises dans l'application, nous avons donc fait en sorte qu'elle soit personnalisable. Pour cela, il suffit d'indiquer lorsqu'on appelle cette fenêtre le texte du message ainsi que le texte des différents boutons.

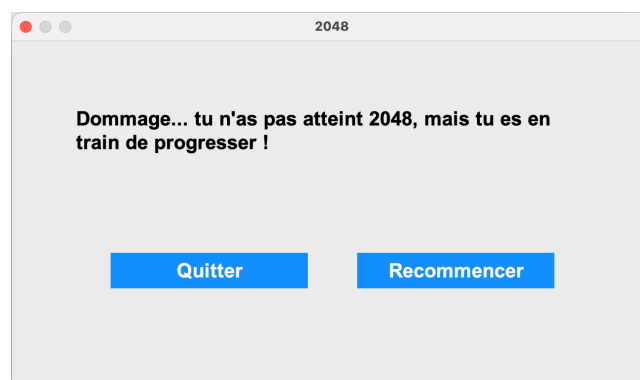


Figure 11 : JDialogPersonalized

4.3. Requêtes BDD et MVC

Ensuite, nous avons implémenté la logique autour des appels vers la base de données. Pour cela, nous nous sommes inspirés du motif d'architecture logicielle destiné aux interfaces graphiques, le MVC ou Modèle-Vue-Contrôleur qui est apparu en 1978. Ce motif est composé de 3 modules ayant chacun une responsabilité différente :

- **Modèle (Model)** : contient les données à afficher
- **Vue (View)** : contient la présentation de l'interface graphique, une fenêtre d'application par exemple
- **Contrôleur (Controller)** : contient la logique concernant les actions de l'utilisateur et modifie les données du modèle et de la vue

Cependant, nous n'avons pas suivi ce motif à la lettre, nous avons plutôt réalisé le motif suivant : Modèle-Service-Répertoire. Voici leur responsabilité :

- **Modèle (Model)** : contient les données à afficher
- **Service** : utilise les modèles pour appeler le répertoire et renvoyer les informations obtenues aux différentes fenêtres
- **Répertoire (Repository)** : se charge d'effectuer les requêtes à la base de données et de renvoyer au service les données obtenues

Nous allons expliquer le fonctionnement de ce modèle avec l'image ci-dessous ([cf. Figure 12](#)). Lorsque le joueur accède à une fenêtre de l'application, le code correspondant à celle-ci est alors exécuté. Dans ce code, il y a notamment la récupération des données depuis la base de données. Pour cela, la fenêtre fait appel à un Service qui va se charger d'appeler le Repository. Ce dernier interroge la base de données et récupère les données nécessaires, puis les transmet au Service. Ensuite, le Service peut effectuer des vérifications ou des transformations sur les données, par exemple, vérifier que le nom d'utilisateur existe bien lorsque le joueur veut se connecter. Les données sont ensuite transmises à la fenêtre qui se charge de les afficher à l'écran comme le meilleur score réalisé par le joueur.

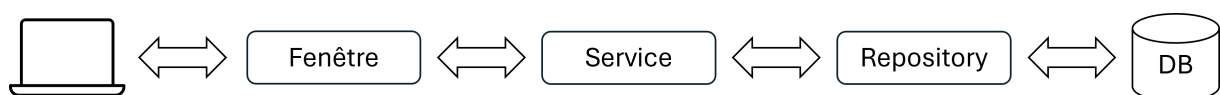


Figure 12 : Modèle utilisé pour l'application

Lorsqu'un Repository effectue un appel à la base de données, la première chose qu'il vérifie est que la connexion est toujours ouverte. En effet, au lancement de l'application, la connexion à la base de données est établie, mais une erreur peut survenir et fermer cette connexion. Si la connexion n'existe plus, un simple message s'affiche dans la console de débogage, nous n'avons pas implémenté une solution qui consisterait à essayer de se reconnecter. Une fois que l'on s'est assuré que la connexion est ouverte, on exécute la requête, puis on récupère les données et elles sont renvoyées au Service qui en a fait la demande, comme vue précédemment.

Après l'ajout des requêtes à la base de données, le jeu est désormais entièrement opérationnel. Le joueur a la possibilité de créer un compte ou de se connecter, de consulter et modifier son profil, ainsi que de jouer une partie dont le résultat est enregistré dans la base de données.

Les éléments suivants se concentrent sur des ajustements visant à améliorer l'expérience utilisateur, faciliter la relecture du code et simplifier le processus de débogage.

4.4. Layout

L'une des premières améliorations a consisté à utiliser les différents **Layout** prédéfinis disponibles avec la bibliothèque Swing. Un Layout est un gestionnaire de disposition qui détermine lui-même la manière de répartir les composants qu'on lui donne selon le type du Layout ainsi que les arguments que chacun prend en compte.

Les Layout nous ont été particulièrement utile pour avoir un joli rendu dans chaque de nos fenêtres et ce, peu importe avec quel écran on utilise l'application. Plus besoin de réaliser des calculs savants pour bien aligner les composants dans la fenêtre de sorte qu'ils soient centrés et avec le même espace entre chacun.

Il y a par exemple le **BorderLayout** (cf. [Figure 13](#)) qui permet de disposer les composants dans 5 zones différentes de l'écran. Pour cela, il suffit de préciser dans quelle zone on veut placer notre composant au sein du Layout.

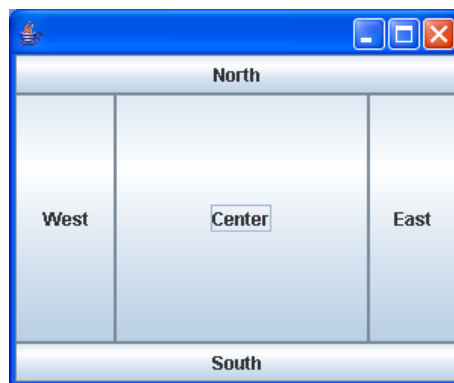


Figure 13 : Illustration du BorderLayout

L'autre avantage des Layout est que l'on peut les emboîter les uns dans les autres. Nous allons détailler le fonctionnement à partir de la fenêtre de jeu où ont été représenté les différents Layout (cf. [Figure 14](#)). La fenêtre contient dans un premier temps un **BorderLayout** en rouge. Puis dans la partie Nord, en violet, on retrouve un **BoxLayout** en configuration horizontale avec l'avatar et le pseudo du joueur ainsi que le bouton pour quitter la partie. Ensuite, la partie centrale du **BorderLayout** est composée d'un **BoxLayout**, en vert, en configuration verticale regroupant le texte 2048 et deux autres Layout. Ces deux derniers Layout sont des **GridLayout**, permettant de placer les éléments sous la forme d'une grille. Le premier, en orange, contient les informations relatives à la partie. Le second, en bleu, permet d'afficher le plateau et les différentes cases qui le constitue.

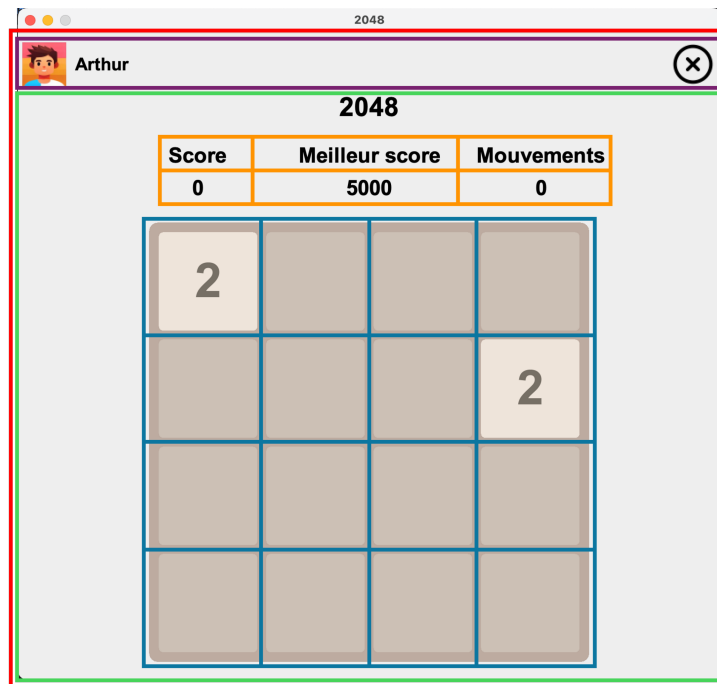


Figure 14 : Fenêtre Game avec Layout

4.5. Factorisation du code

Ensuite, nous avons repris le code réalisé jusque-là afin de voir si des parties du code étaient répétées et ainsi créer une classe générique pour éviter la répétition de code, de sorte à simplifier la relecture du code.

Tout d'abord, il y a la classe « **ImageIconPersonalized** » dont le but est de récupérer une image et de la redimensionner pour pouvoir afficher une image à l'écran. Il est important de redimensionner les images car elles ne font pas toutes la même taille initiale. Cela permet donc d'avoir des images nettes.

Ensuite, on va retrouver trois classes permettant de réaliser les différents boutons :

- « **JButtonPersonalized** » pour créer les boutons bleus visibles par exemple sur la page: Login
- « **JButtonProfileGame** » pour créer les boutons se trouvant en bas des fenêtres: Profile et: ProfileGame
- « **JButtonWithIcon** » pour créer les boutons possédant une image à la place d'un texte, comme le bouton pour quitter la partie sur la page: Game

Enfin, nous avons créé deux classes pour afficher les différents textes présents dans l'application :

- « **JLabelPersonalizedForGridLayout** » pour les textes présents dans les **GridLayout**
- « **JLabelPersonalized** » pour tous les autres textes

Nous avons été obligés d'utiliser deux classes différentes pour plus de clarté et pour ne pas surcharger le code.

4.6. Classe Utils

Puis, nous avons créé une classe « **Utils** » qui contient de nombreux éléments qui sont utilisés plusieurs fois dans le code.

Nous avons d'abord géré tout ce qui se réfère aux dimensions pour faire en sorte d'avoir une application qui s'adapte à n'importe quel écran. Pour cela, on récupère les dimensions de l'écran sur lequel le joueur utilise l'application qui nous permette de définir la dimension de chaque fenêtre. On utilise également les dimensions de l'écran de manière à positionner la fenêtre au centre de l'écran. De plus, avec les dimensions de la fenêtre que nous avons calculées précédemment, nous pouvons définir la largeur et la hauteur du plateau ainsi que des cases qui le constituent.

Ensuite, nous avons défini des variables ainsi que des méthodes relatives aux couleurs et aux textes utilisées dans la fenêtre de jeu. Il y a un dictionnaire permettant de récupérer la couleur de la case en fonction de la valeur de la tuile, par exemple la tuile 2048 aura une couleur de fond dorée. Nous avons une autre méthode pour récupérer la taille de la police selon la valeur de la tuile, car plus la valeur augmente, plus le texte doit être écrit petit pour qu'il s'affiche correctement dans la case du plateau.

4.7. Introduction de la langue

Pour finir, nous avons ajouté une dernière fonctionnalité pour terminer l'application : la possibilité de changer la langue avec la classe « **TranslationManager** ».

Le fonctionnement est assez simple, on utilise un dictionnaire par langue avec comme clé, la description de l'élément et comme valeur, le texte dans la langue associée. Par exemple, pour le bouton **Login** présent sur la page Home, la clé dans tous les dictionnaires sera « **home.login.button** » et la valeur sera :

- **Login** : pour la langue anglaise
- **Connexion** : pour la langue française

Il est important de bien définir la clé de façon très concrète, pour bien identifier à quel élément elle se réfère.

Bien que le fonctionnement soit simple, la tâche n'en est pas moins longue et chronophage. En effet, cela demande beaucoup de temps de traduire chaque texte dans les différentes langues que l'on souhaite avoir dans notre application. Heureusement, nous n'avons utilisé que deux langues.

5. Architecture du projet

Nous allons rapidement détailler l'architecture du projet tel qu'il apparaît lorsqu'on ouvre le projet avec IntelliJ (*cf. Figure 15*).

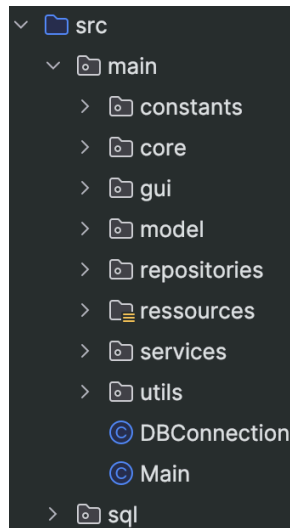


Figure 15 : Architecture du projet

Le premier dossier « **main** » qui contient tous les fichiers nécessaires au fonctionnement de l'application. A l'intérieur on retrouve deux fichiers :

- **DBConnection** : contient le code pour établir et fermer la connexion à la base de données
- **Main** : contient le code pour lancer l'application

Il y a également huit dossiers :

- **constants** : contient deux fichiers de classe « **enum** » permettant de définir un ensemble de constantes représentant des valeurs fixes, comme les jours de la semaine
- **core** : contient les quatre fichiers implémentant les classes présentes dans le diagramme de classe (*cf. Figure 2*)
- **gui** : contient les huit fichiers représentant les fenêtres de l'application introduites dans la partie **Fenêtres**
- **model** : contient les fichiers permettant de récupérer les données de la base de données pour les utiliser dans l'application
- **repositories** : contient les Repository chargés d'interroger la base de données
- **ressources** : contient toutes les images utilisées dans l'application
- **services** : contient les Service qui réalisent des traitements sur les données
- **utils** : contient les fichiers implémentant les classes de la partie **Classe Utils**

Le second dossier « **sql** » contient les différents fichiers permettant de créer, remplir et supprimer les tables utilisées par l'application.

6. Difficultés rencontrées

Nous allons finir par évoquer les différentes difficultés que nous avons pu rencontrer durant ce projet.

La principale et première difficulté rencontrée est survenue lorsque nous avons développé la logique de déplacement et fusion des cases. Étant la fonctionnalité la plus importante du jeu, il est important que celle-ci soit bien implémentée pour que l'on puisse jouer au jeu. Nous avons donc commencé à étudier le mécanisme en jouant au jeu sur une des versions disponibles en ligne. Il nous a fallut ensuite plusieurs essais pour arriver à un code fonctionnel.

Une autre difficulté a été de placer à la main tous les éléments à l'intérieur de chaque fenêtre. Plus on met d'éléments dans une fenêtre, plus les calculs sont compliqués pour faire en sorte d'espacer et de centrer correctement les éléments les uns par rapport aux autres. C'est la raison pour laquelle nous avons décidé de chercher des alternatives et nous avons trouvé les Layout. Les Layout nous ont simplifié le développement et nous ont fait gagner beaucoup de temps. De plus, le rendu final est plutôt joli.

La dernière difficulté consistait à utiliser notre base de données directement depuis le code Java sous IntelliJ. Pour cela, nous avons dû trouver comment ajouter à notre projet le driver nécessaire. Puis, nous avons cherché dans la documentation comment écrire les différents codes permettant d'établir et de fermer la connexion mais également le code pour interroger et récupérer les données.

Conclusion

Ce projet de développement du jeu 2048 en Java a été une expérience enrichissante, nous permettant d'explorer les multiples facettes de la conception logicielle et du développement d'interfaces graphiques. Nous avons pu réaliser une application en utilisant le modèle MVC qui est le plus répandu à l'heure actuelle. De plus, l'utilisation de GitHub nous a permis de nous familiariser avec le principe de gestionnaire de versions, système utilisé par tous les développeurs. L'utilisation d'une base de données, que nous avons dû connecter à notre code, est une bonne chose, cela nous a fait voir les deux facettes front-end et back-end.

Les difficultés rencontrées, telles que la gestion de la logique de déplacement des cases ou l'intégration de la base de données, ont été des défis stimulants qui nous ont permis de renforcer nos compétences techniques et notre capacité à résoudre des problèmes complexes.

En conclusion, ce projet a non seulement permis de consolider nos connaissances en programmation mais nous a fait comprendre l'importance de l'expérience utilisateur et de l'adaptabilité des applications. Ces apprentissages constituent une base solide pour travailler dans des métiers liés au développement.

Bibliographie

- [1] « IntelliJ IDEA » : <https://www.jetbrains.com/fr-fr/idea/>
- [2] « JetBrains » : <https://www.jetbrains.com/fr-fr/>
- [3] « GitHub » : <https://github.com/>
- [4] « Git » : <https://git-scm.com/>
- [5] « Swing » : <https://docs.oracle.com/javase/8/docs/api/???javax/swing/package-summary.html>
- [6] « PostgreSQL » : <https://www.postgresql.org/>
- [7] « pgAdmin » : <https://www.pgadmin.org/>